

2026—2027 学年第一学期

第 2 章 神经网络

Neural Networks

- 《深度学习》课程 · 课时一：前馈神经网络 (FFN)
- 理论 2 学时 + 实践 1 学时 · 全章共 3 学时

授课教师：李家琦

回顾：第一章埋下了什么？

Recap

- **深度学习定义**：多层神经网络自动学习特征表示
- **第一章的“单层特例”**：感知机——只有输入直接到输出，没有隐藏层
- **遗留问题**：异或 XOR 单层切不开，怎么办？
- **本章任务**：把“单层”升级为“多层”，把“特例”升级为“一般”

◆ 第一章讲的是“单层特例”，第二章讲的是“多层一般”——这就是 FFN。

本章学习路径：三个课时

Chapter Roadmap



◆ 学完本章：你不仅知道 FFN 长什么样，还知道它怎么学、怎么搭。

01

从神经元到数学单元

From Neurons to Units

- 你的大脑：860 亿个神经元，功耗仅 20 瓦
- 神经元怎么传递信息？
- 把生物学“翻译”成数学

课堂讨论

2 分钟

大脑比超级计算机慢百万倍，为什么你能瞬间认出一张脸？

- ▶ 超级计算机算力是人脑的多少倍？
- ▶ 人脑“算”人脸识别快在哪？

提示：关键差异不是速度，而是架构：人脑是千亿级“处理器”并行工作，且能耗极低。



生物神经元的三个部件

Biological Neuron

- **树突 Dendrite**: 接收其他神经元传来的电信号——相当于“输入端”
- **细胞体 Soma**: 汇总所有输入信号, 超过阈值就“兴奋”——相当于“处理器”
- **轴突 Axon**: 把兴奋信号传给下一个神经元——相当于“输出端”
- **记住三个部件: 树突收、胞体算、轴突发**

数据对比

- 人脑神经元 约 860 亿个
- 连接点(突触) 约 100 万亿个
- 整机功耗 约 20 瓦

◆ 上百亿个简单单元连成网络, 涌现出智能。

信号如何跨越神经元之间的“河”

Crossing the Synapse



- ◆ 信号跨过突触靠化学递质；递质越多、突触越强，影响越大。

突触传递：信号强度由谁决定？

Synaptic Strength

生物突触

- 递质多 → 下一个神经元被激发越强
- 连接强度不同 → 信号被“加权”
- 突触强度可塑性：用得越多，连接越强

VS

数学翻译

- 突触强度 → 权重 w
- 递质多少 → 输入值 x 乘以 w
- 这就是“权重”这个术语的生物学来源

◆ 赫布法则：一起激活的神经元连在一起——权重来自经验，不是天生。

生物学 → 数学：完整对照表

Biology to Math

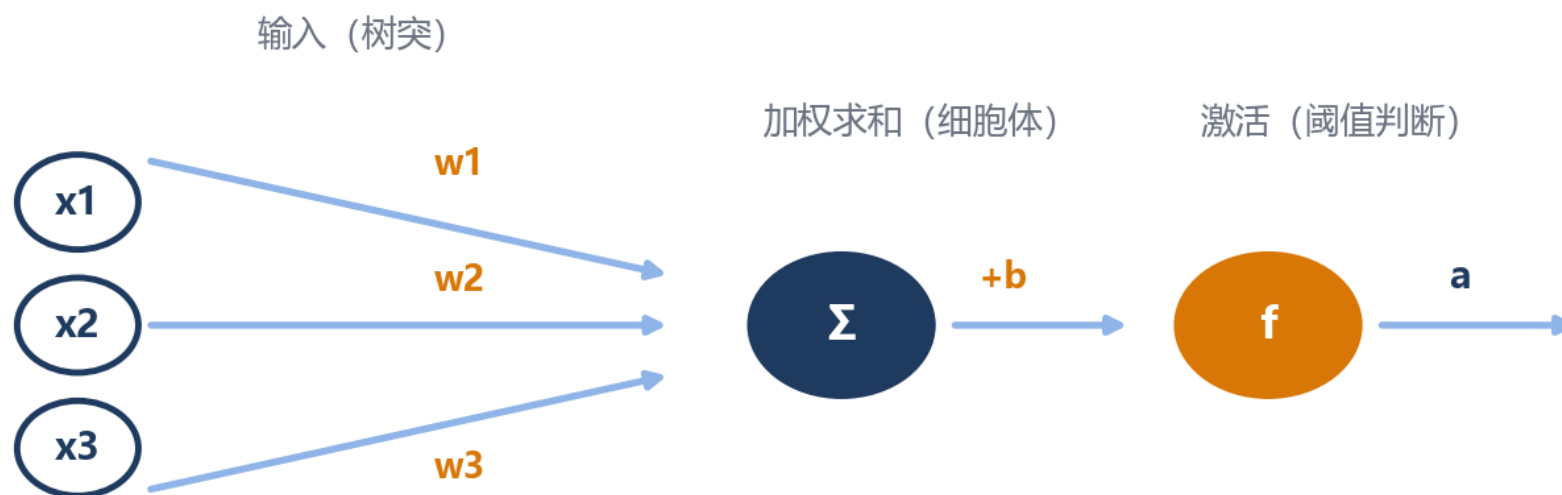
生物部件	功能	数学模型	符号
树突 Dendrite	接收其他神经元的信号	输入向量	$x_1, x_2, \dots, x_n$
突触强度	信号放大的倍数	权重	$w_1, w_2, \dots, w_n$
细胞体 Soma	汇总所有输入信号	加权求和 + 偏置	$z = \sum w_i x_i + b$
轴突 Axon	把信号传给下一层	激活函数输出	$a = f(z)$

◆ 这张表就是“把生物学翻译成数学”的完整答案——FFN 的每个单元都从这里来。

一个人工神经元：收、算、发

One Artificial Neuron

一个人工神经元的完整解剖：收、算、发



$$z = w1 \cdot x1 + w2 \cdot x2 + w3 \cdot x3 + b$$

$$a = f(z) \leftarrow \text{这一步给网络注入非线性}$$

- ◆ 树突收、细胞体算、轴突发——翻译成数学就是：加权求和 + 激活。

前馈神经网络：从单个到一群

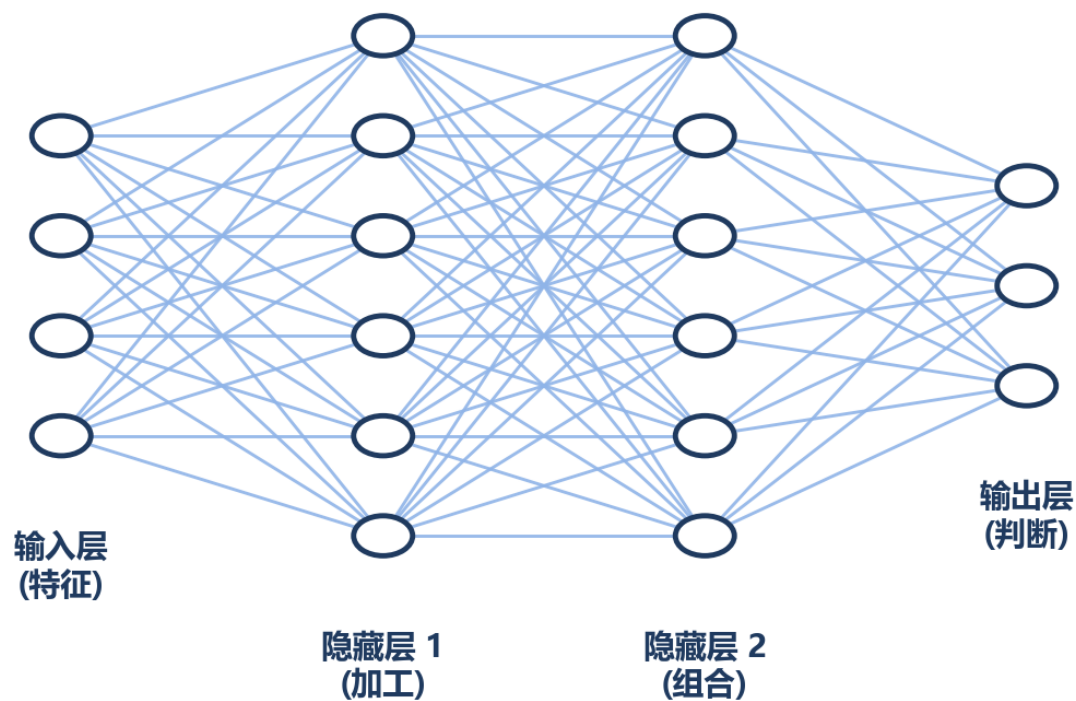
Feedforward Neural Networks

- 单个单元 → 层层堆叠的“加工流水线”
- FFN 的三要素：输入层、隐藏层、输出层
- 前向传播：数据在网络中的单向旅程

FFN 的骨架：输入层 · 隐藏层 · 输出层

FFN Architecture

多层神经网络：每一根线都是一个“可学习的权重”



♦ FFN = 层层加权求和 + 激活；数据从输入层单向流到输出层——“前馈”二字由此而来。

FFN 的数学表达：一个复合函数

FFN as a Function

- 单层（感知机）： $y = f(w \cdot x + b)$
- 两层 FFN： $y = f_2(W_2 \cdot f_1(W_1 x + b_1) + b_2)$
- 三层 FFN：函数套函数，一层套一层
- FFN 本质：一个巨大的复合函数——这也是“深度”的数学含义

◆ 层数越深，复合函数的嵌套越深——这就是“深”的数学来源。

感知机 = 单层 FFN (历史特例)

Perceptron = 1-Layer FFN

第一章的感知机

- 输入 → 输出, **没有隐藏层**
- 只能画一条直线
- 能力天花板: XOR 切不开

VS

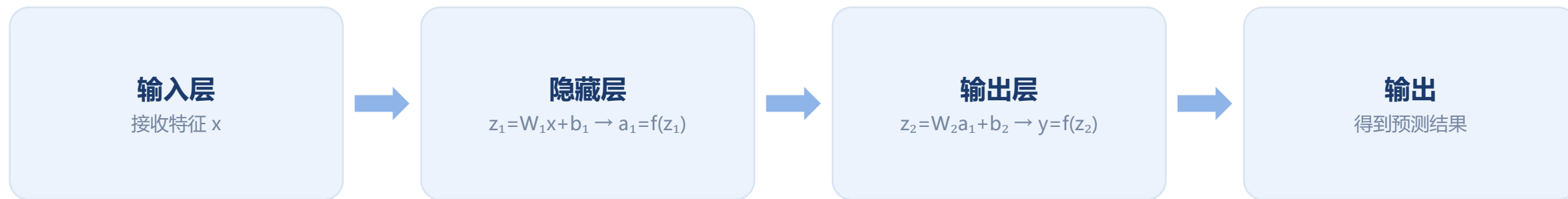
FFN 的一般形式

- 输入 → 隐藏层 → 输出, **可以有若干隐藏层**
- 能画任意复杂的决策面
- 感知机是 FFN 在隐藏层为 0 时的特例

◆ 划重点: 感知机不是另一种模型, 它就是隐藏层为 0 的 FFN。

前向传播：数据在网络中的单向旅程

Forward Pass



- ◆ 前向传播 = 数据从输入层一路算到输出层；每一步都是“加权求和+激活”。

FFN 的能力：为什么要有隐藏层

Capability of FFN

- 隐藏层 = 特征加工车间
- 万能逼近：足够宽就能贴任意曲线
- XOR 的答案：两层 FFN 轻松切开

隐藏层：网络的“特征加工车间”

Hidden Layer as Feature Workshop

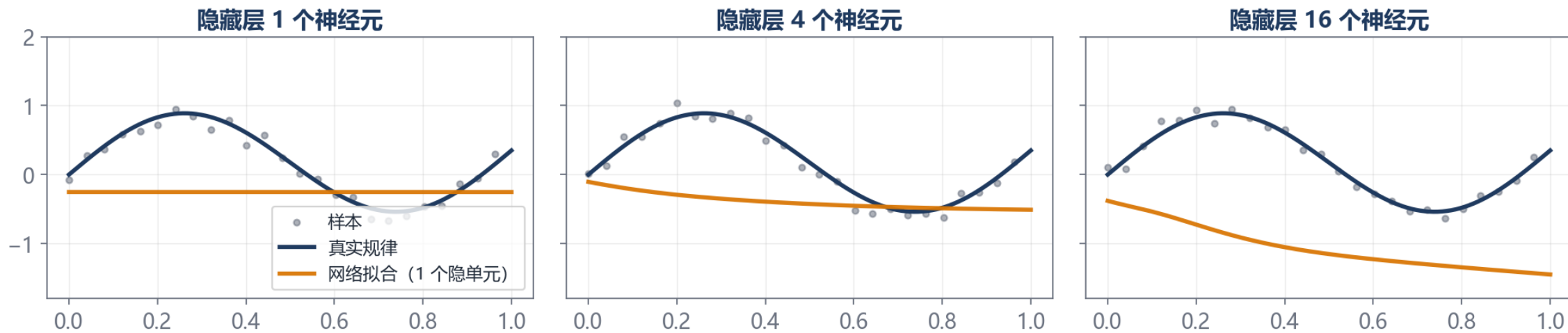
- **第一层：**从原始输入提取“初级特征”（边缘、局部）
- **第二层：**把初级特征组合成“高级特征”（部件、形状）
- **输出层：**基于高级特征做最终判断
- **层数越深，特征的抽象层次越高——这就是第一章“表示学习”的实现**

◆ **隐藏层逐层加工：**像素→边缘→部件→物体。第一层学到的，是下一层的“原料”。

万能逼近：足够多的隐单元能贴出任意的弯

Universal Approximation

万能逼近定理：隐单元越多，网络能“贴”出的曲线越自由



◆ 隐单元越多，FFN 的表达力越强——但参数也多，过拟合风险上升。

没有激活函数，多深都等于一层

Why Nonlinearity Matters

如果去掉激活函数

- 两层网络: $z = W_2(W_1x + b_1) + b_2$
- 展开: $z = (W_2W_1)x + (W_2b_1 + b_2)$
- **仍是一个线性模型!**
- 叠 100 层 = 叠 1 层，网络毫无意义

VS

加上非线性激活后

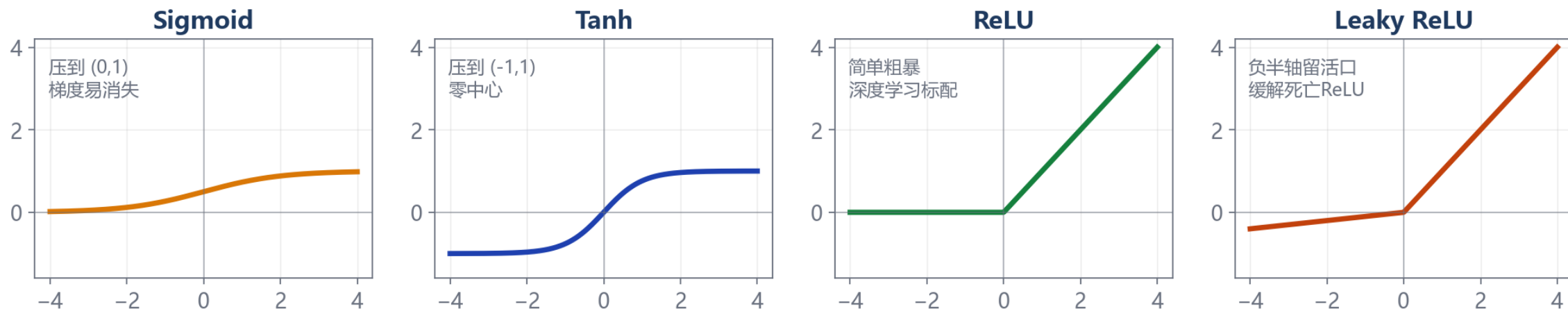
- 每一层都是一次“弯折”空间
- 层数越深，能表达的形状越复杂
- 线性可分 $\rightarrow$ 任意复杂的决策面
- 非线性 = FFN 表达能力的来源

◆ 没有激活函数，FFN 退化成一条直线；激活函数是“深”得以成立的前提。

四个必会的激活函数：一眼看懂它们的脾气

Activation Functions

四个常用激活函数：非线性是“深”的全部魔法



- ◆ Sigmoid 压到(0,1)、Tanh 零中心、ReLU 简单高效、Leaky ReLU 给负半轴留活口。

课堂讨论

2 分钟

如果是你，会怎么选激活函数？

- ▶ 做二分类的输出层（猫/狗），用 Sigmoid 还是 ReLU？为什么？
- ▶ 隐藏层用 Sigmoid 训练 50 层的网络，可能遇到什么问题？

提示：输出层看任务需求（概率→Sigmoid）；隐藏层看重梯度流动性（深网络→ReLU 系）。



FFN 能“算”：逻辑门与 XOR 的答案

FFN and Logic Gates

- 单层 FFN（感知机）能实现 AND/OR/NOT
- XOR：单层切不开，两层轻松解决
- 隐藏层的价值：把数据“揉”到可分

单层 FFN 能“算”：实现逻辑门

Logic Gates with 1-Layer FFN

逻辑门	输入→输出	一组可行权重 (w_1, w_2, b)
AND	$(0,0) \rightarrow 0$ $(1,1) \rightarrow 1$	$w_1=w_2=1, b=-1.5$
OR	$(0,0) \rightarrow 0$ 其余 $\rightarrow 1$	$w_1=w_2=1, b=-0.5$
NOT	$x \rightarrow 0$ 则 1; $x \rightarrow 1$ 则 0	$w=-1, b=0.5$
NAND	AND 取反	$w_1=w_2=-1, b=1.5$

◆ 单层 FFN 是可编程的：改权重偏置，就能实现不同的逻辑运算。

手推验证：AND 门的四次输入

Walkthrough: AND Gate

1 (0,0)

$$z = 1 \times 0 + 1 \times 0 - 1.5 = -1.5 < 0 \rightarrow \text{输出 } 0 \checkmark$$

2 (0,1)

$$z = 1 \times 0 + 1 \times 1 - 1.5 = -0.5 < 0 \rightarrow \text{输出 } 0 \checkmark$$

3 (1,0)

$$z = 1 \times 1 + 1 \times 0 - 1.5 = -0.5 < 0 \rightarrow \text{输出 } 0 \checkmark$$

4 (1,1)

$$z = 1 \times 1 + 1 \times 1 - 1.5 = 0.5 > 0 \rightarrow \text{输出 } 1 \checkmark$$

5 结论

四次全对！一组权重实现 AND——单层 FFN 的“知识”全在权重里

♦ 手推一次就明白：FFN 的“知识”全在权重和偏置里。

真实走一遍：AND 门的完整真值表

AND Gate Table

输入 x_1	输入 x_2	$z = x_1 + x_2 - 1.5$	输出 ($z > 0?$)	目标
0	0	$0 + 0 - 1.5 = -1.5$	0 ✓	0
0	1	$0 + 1 - 1.5 = -0.5$	0 ✓	0
1	0	$1 + 0 - 1.5 = -0.5$	0 ✓	0
1	1	$1 + 1 - 1.5 = 0.5$	1 ✓	1

♦ 权重 (1,1,-1.5) 的 AND：每一行都代进去算一遍，四次全对。

互动：亲手“编”一个逻辑门

Lab: Logic Gates

► 互动实验台

实验台① 神经元逻辑门

文件：..\交互实验台\01_神经元逻辑门.html

双击打开即可互动 · 无需联网

课堂怎么玩

- 1 点 AND/OR 预设，看决策线和真值表全对
- 2 手动拖权重和偏置，理解“调参数=改行为”
- 3 挑战：调出 XOR——你会失败，这正是单层 FFN 的天花板

看什么：调权重=改行为；XOR 调不出来=单层 FFN 需要隐藏层

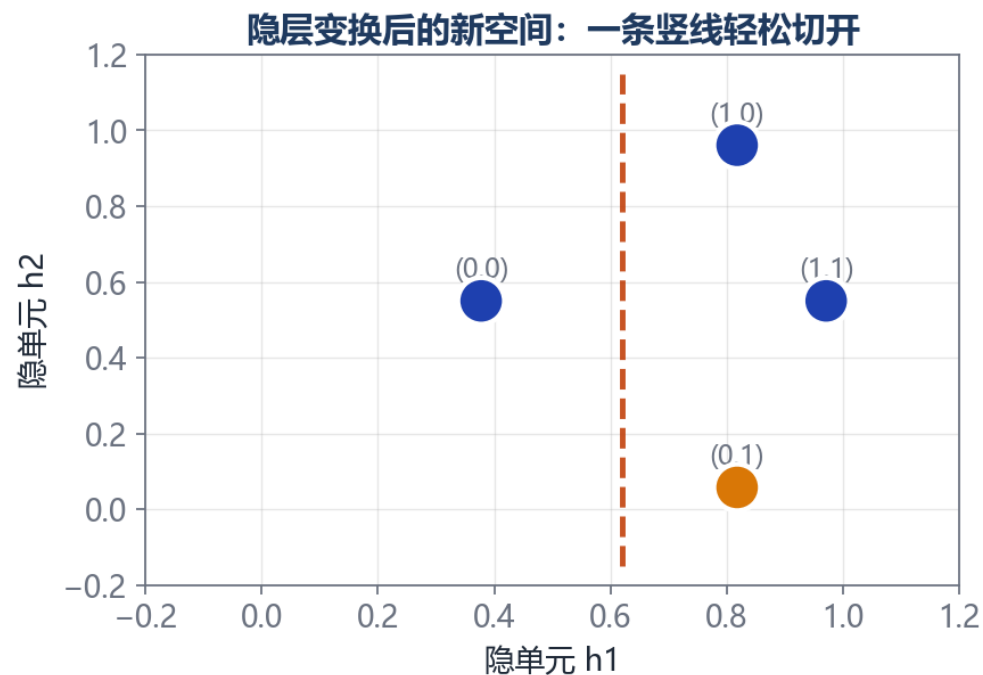
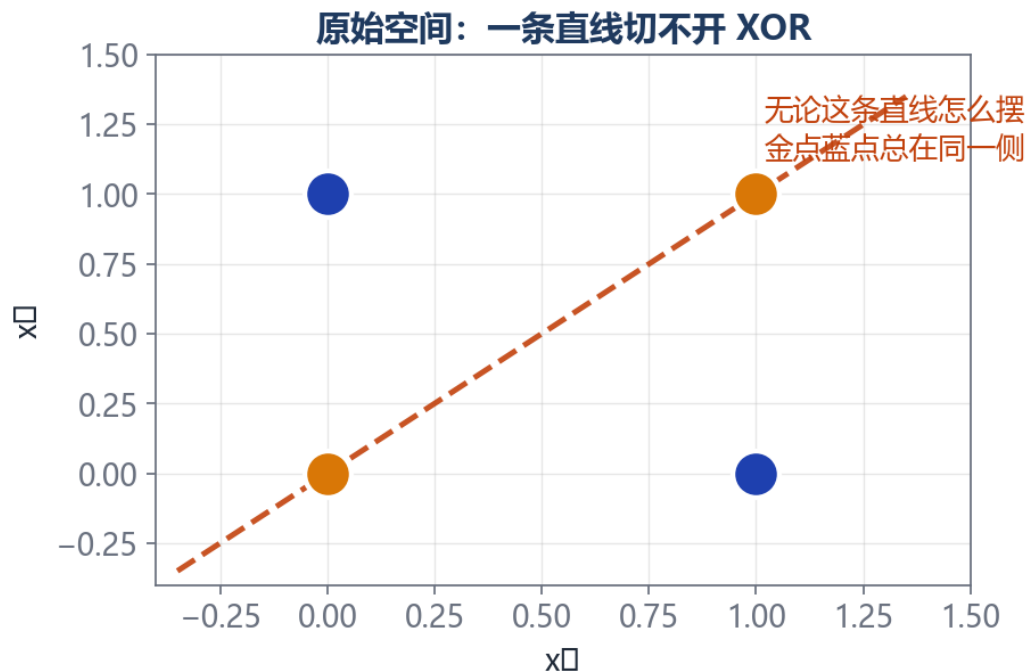
XOR: 单层 FFN 的天花板

四个点: $(0,0)(1,1)$ 一类, $(0,1)(1,0)$ 另一类——任何一条直线都切不开

隐藏层的答案：把数据“揉”到可分

Hidden Layer Transforms Space

隐藏层在干什么：把数据“揉”成线性可分的形状（1969 年死穴的解药）



◆ 全章最重要的一张图：左边切不开，右边加隐藏层后一条竖线切开——隐藏层在换“看法”。

互动：隐藏层如何“揉”数据

Lab: XOR with Hidden Layer

► 互动实验台

实验台③ 隐藏层与 XOR

文件：..\交互实验台\03_隐藏层与XOR.html

双击打开即可互动 · 无需联网

课堂怎么玩

- 1 看原始空间四个点，确认一条线切不开
- 2 加入隐藏层，观察数据点被“揉”到新位置
- 3 在新空间里，一条竖线轻松分开两类

看什么：隐藏层 = 数据换坐标系；换完坐标系，不可分变可分

从第一章到第二章：特例 → 一般

第一章：深度学习是什么（感知机作为单层特例登场）

第二章：FFN 是什么（把单层升级为多层）

第三章预告：FFN 遇上图像 = 卷积神经网络

动手推导：为什么“没激活=白深”

Derivation: Why Nonlinearity

1 两层网络

$$a_1 = W_1x + b_1, y = W_2a_1 + b_2$$

2 代入展开

$$y = W_2(W_1x + b_1) + b_2$$

3 合并系数

$$y = (W_2W_1)x + (W_2b_1 + b_2)$$

4 关键观察

W_2W_1 仍是矩阵，整体仍是 x 的线性函数！

♦ 跟着推导一遍：非线性激活不是可选项，是“深”的数学前提。

FFN 能做什么：手写数字识别

Case: Digit Recognition

问题设定

- 8×8 手写数字图 (64 像素)
- 输入层 64 个单元
- 输出层 10 个单元 (0-9 各一个)
- 这是 FFN 的经典入门任务

VS

结构映射

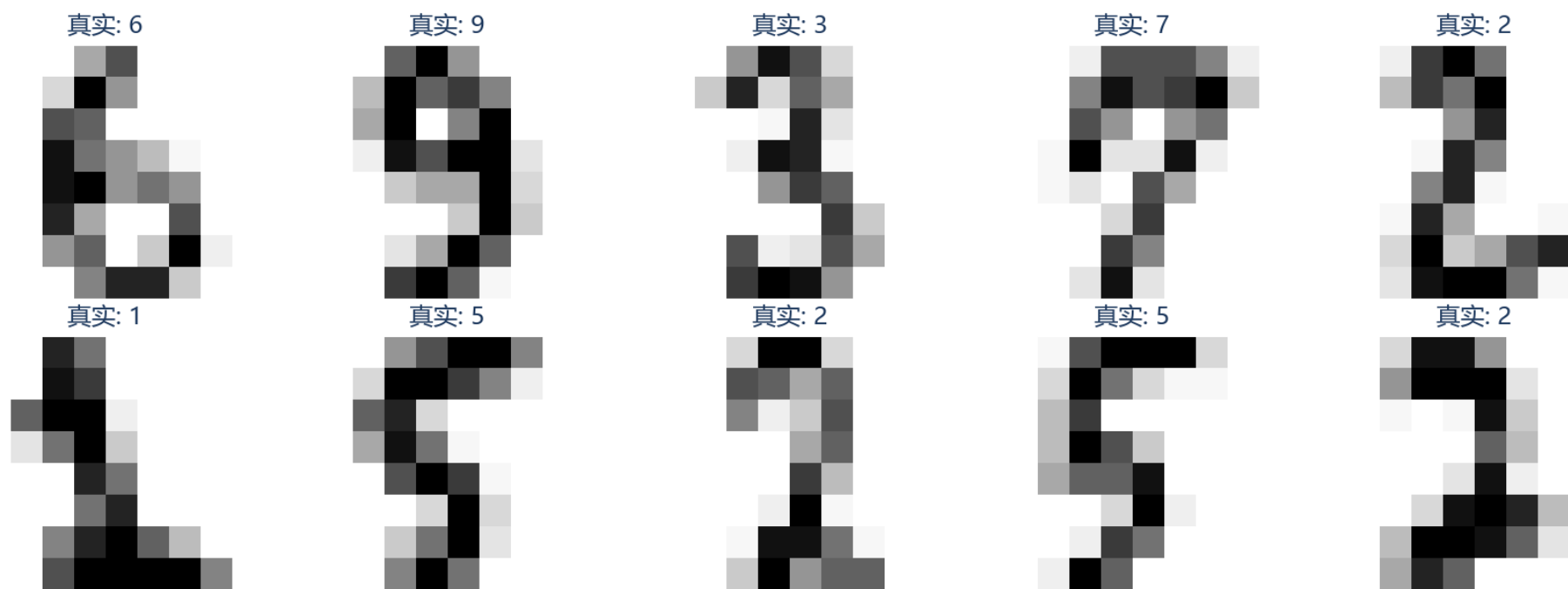
- 64 → 32 → 10 的 FFN
- 隐藏层 32 个单元加工特征
- 输出层 10 个概率，取最大
- 课时三会亲手实现它！

◆ FFN 不是抽象玩具：64→32→10 的结构，就是手写数字识别的完整方案。

真实样例：64 像素的手写数字长什么样

Real Digit Samples

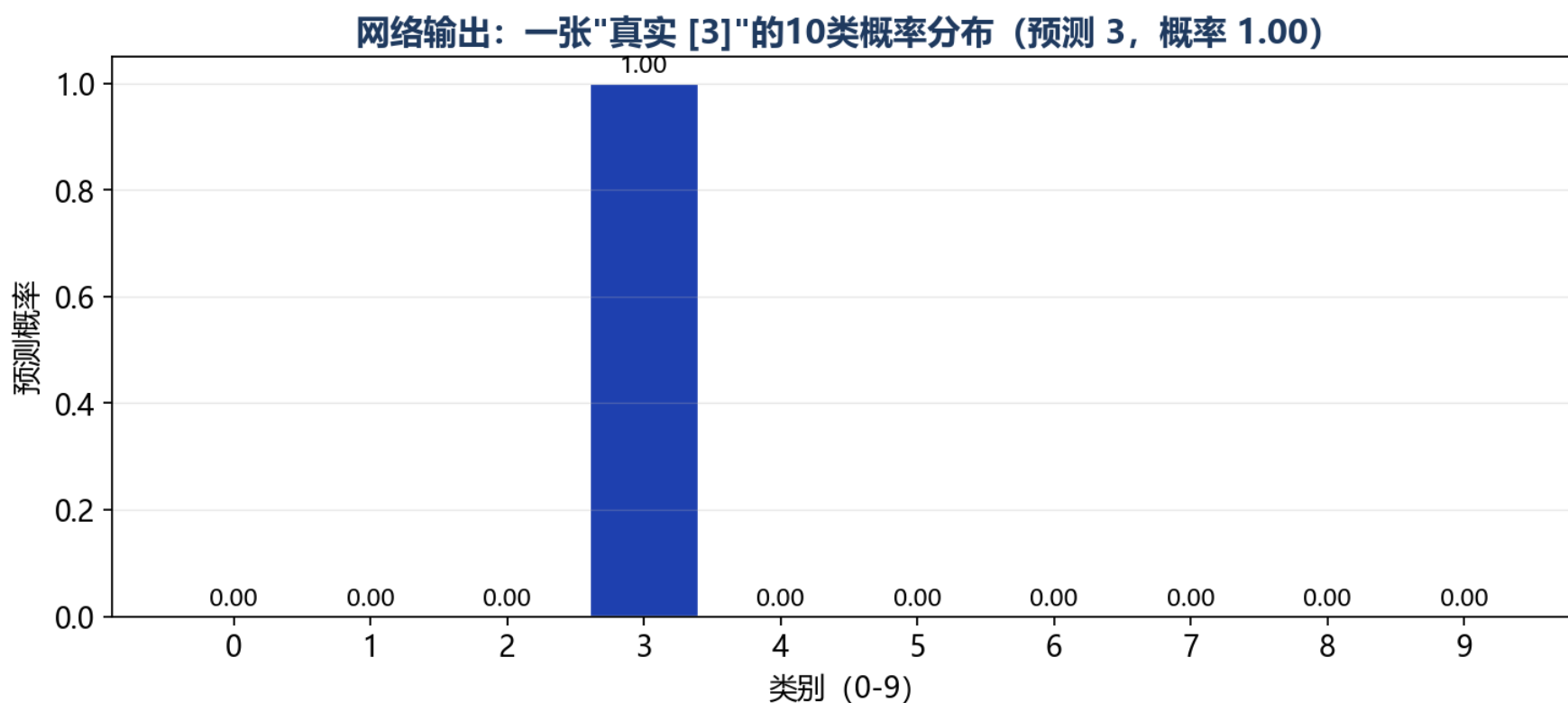
真实输入样例：sklearn 手写数字（8×8 灰度，64 像素 → 输入层 64 个单元）



- ◆ 输入层 64 个单元，接收的就是这样一张 8×8 灰度图的 64 个像素值。

真实输出：网络如何“判断”一个数字

Model Output Probabilities



- ♦ 输出层 10 个单元，每个给一个概率——网络不是“说”这是几，而是给 10 个可能性的打分。

偏置 b : 决策边界的“平移旋钮”

The Role of Bias

没有偏置 ($b=0$)

- 决策线永远过原点
- 只能分“过原点”的两类
- $(0,0)$ 永远输出 0
- 能力受限

VS

有偏置 ($b \neq 0$)

- 决策线可以自由平移
- 能分“任意位置”的两类
- 偏置 = 决策线的“截距”
- 必须作为可学习参数

♦ 权重决定方向，偏置决定位置——FFN 的每个单元都有这两个自由度。

FFN 全景：一条线串起今天

FFN Panorama



♦ FFN 全景：输入→加工→非线性→输出；怎么让输出变准？→ 课时二 BP 见。

随堂练习

4 分钟

FFN 结构小测

- ▶ 画出 “两层 FFN (输入2→隐藏3→输出1)” 的草图，标出层和连接
- ▶ 为什么没有激活函数时，两层 FFN 退化成线性模型？
- ▶ XOR 为什么单层切不开、两层可以？

提示：每根连接是一个权重；激活函数提供非线性；隐藏层变换坐标空间。



课时一总结：FFN 的完整画像

Lesson 1 Summary



单元

加权求和 + 激活；收、算、发对应输入、权重、输出



结构

输入层→隐藏层→输出层，层间全连接，前向单向流动



数学

FFN = 复合函数，层数 = 嵌套深度；感知机是 0 隐藏层特例



能力

隐藏层加工特征；非线性保证表达力；万能逼近



XOR

单层切不开，两层轻松解决——隐藏层的价值证明

◆ 课时二：让 FFN 学会自己找权重——反向传播登场。

课时一引用的经典文献

References

McCulloch & Pitts, 1943

A Logical Calculus of the Ideas Immanent in Nervous Activity

神经元的第一个数学模型（M-P 神经元），FFN 的起点

Hebb, 1949

The Organization of Behavior

“同时激活的神经元连在一起”——权重来自经验的生物学基础

Rosenblatt, 1958

The Perceptron: A Probabilistic Model...

感知机：单层 FFN 的开山之作（第一章已见）

Hornik et al., 1989

Multilayer Feedforward Networks are Universal Approximators

万能逼近定理：FFN 表达力的形式化证明

Minsky & Papert, 1969

Perceptrons

单层 FFN 的局限（XOR），促成隐藏层与多层 FFN 的发展

课时一课后小任务

After Lesson 1

- **任务一：**纸上手推 AND 门四次输入
- **任务二：**交互实验台亲手调权重，观察决策线变化
- **任务三（思考题）：**XOR 明明切得开，两层 FFN 的权重该怎么找？——课时二的预告
- **任务三想不通很正常——它就是反向传播要解决的难题**

动手巩固（30 分钟）

- 手推 AND 门的 4 次输入，确认输出全对
- 打开实验台①，把 OR 调成 AND、尝试调 XOR
- 预习：为什么“两层 FFN”就能切开的 XOR，权重怎么找到？

◆ 课时一作业：结构已懂，任务三的困惑正是课时二的入口。

课时一 · 到此结束

休息 5 分钟 · 课时二将进入反向传播 BP 算法

课时二 · 核心之问

网络错了，谁来负责？每一层怎么分摊？

——反向传播算法：深度学习的引擎，今天亲手拆开它

课时一解决了「长什么样」，课时二解决「怎么学会」

FFN 的骨架已立：结构、前向、能力、XOR 答案
但权重从哪来？——反向传播 BP 算法

起点：把“学习”翻译成“求导”

Loss as a Function

- **误差（损失）函数 L** ：给“网络错得多离谱”打一个分数
- **关键洞察**： L 是所有权重 w 的函数 $\rightarrow$ 可以对 w 求导
- **导数的含义**： w 往哪个方向动一点， L 会变大还是变小、变多少
- **梯度下降**：沿着“负梯度”方向小步走，误差就下降

目标

- 定义误差函数 L
- 求 L 对每个 w 的导数
- 导数指向“该往哪调”

◆ **学习 = 在误差地形里下山；梯度告诉你下山的方向和坡度。**

BP 的两大任务：算梯度 + 用梯度更新

Two Jobs of BP

任务一：算出梯度

- 前向算出损失 L
- 反向算出 $\partial L / \partial w$ (每个权重的“坡度”)
- 这是 BP 的核心贡献

VS

任务二：用梯度更新

- $w \leftarrow w - \eta \cdot \partial L / \partial w$
- 学习率 η 决定步幅
- 梯度只给方向, η 给步幅

◆ BP 本质：前向算损失，反向算梯度，再用梯度更新——循环到收敛。

为什么梯度是“最陡方向”？

Why Gradient is Steepest

- 损失 = 山坡高度，参数 = 你在山上的位置
- **梯度 ∇L 指向**上升最陡的方向
- **所以用 $-\nabla L$ ：**下降最陡的方向
- η 是步幅：太小磨蹭，太大翻车
- **梯度下降 = 站在山坡上，朝最陡下坡方向迈一小步，反复**

◆ 理解梯度是“方向”，学习率是“步幅”，训练就是一个“朝最陡下坡走”的循环。

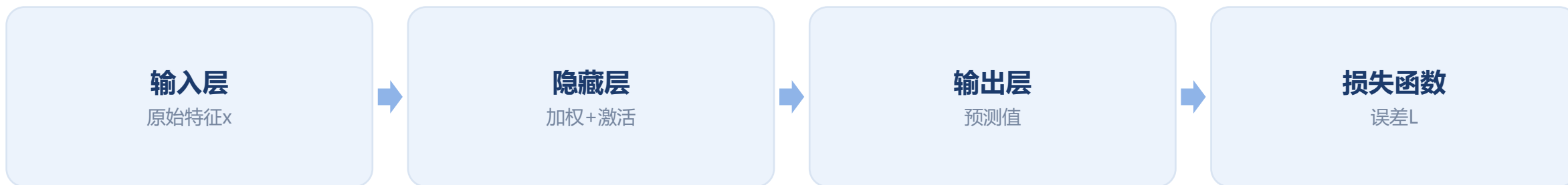
数学工具只有一个

链式法则：复合函数的导数 = 导数的乘积

若 z 依赖 y 、 y 依赖 x ，则 $\partial z / \partial x = \partial z / \partial y \times \partial y / \partial x$ —— 高中学过的知识点，撑起了万亿参数模型的训练

前向传播：数据在网络中的旅程

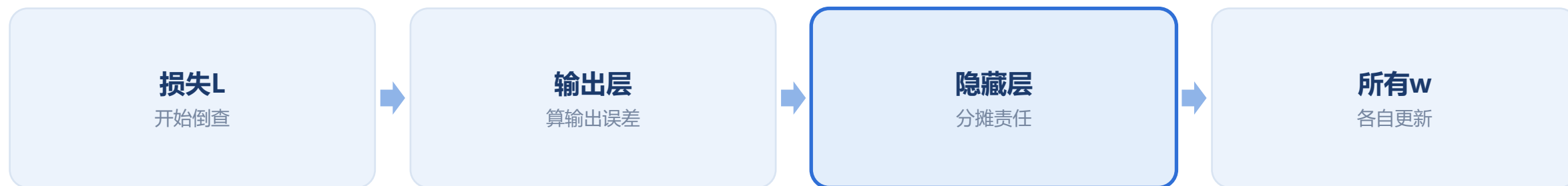
Forward Pass



- ◆ 前向传播 = 从左到右算一遍，终点是“误差分数 L”。

反向传播：误差的“责任倒查”

Backward Pass



- ◆ 误差信号从右向左逐层回传，每层按“责任份额”分摊、各自修正。

直觉类比：工厂的质量追责

An Intuitive Analogy

- **前向 = 生产**：原料(输入)经过车间一、车间二变成成品(预测)
- **质检 = 损失**：成品不合格，不合格程度 = 误差 L
- **追责 = 反向传播**：按各车间对缺陷的“贡献度”分摊责任
- **整改 = 参数更新**：每个工位(权重)按自己的责任大小调整工艺
- 千百次“生产-质检-追责-整改”循环，工厂良品率持续提升

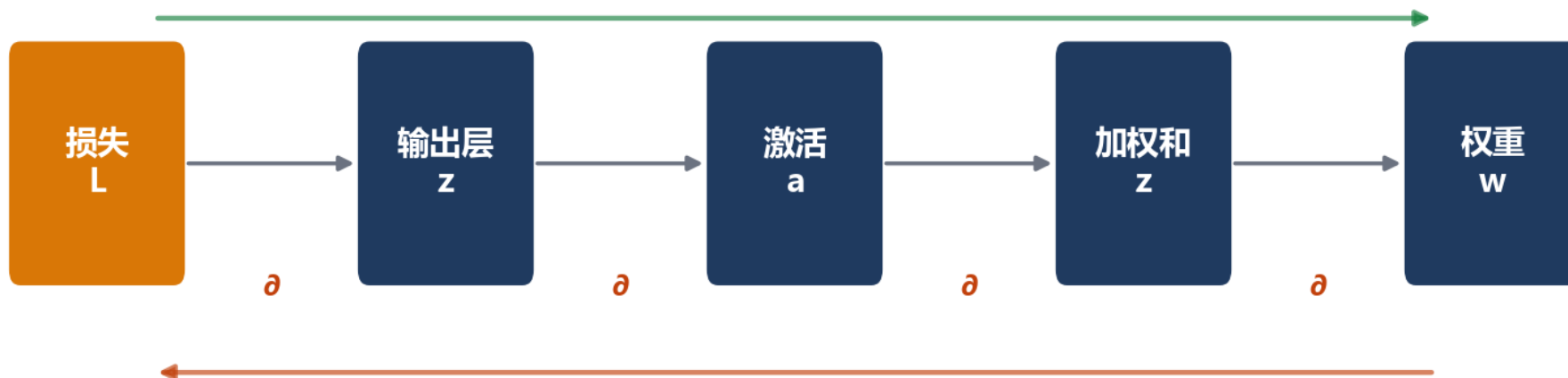
◆ **反向传播 = 一套自动化的质量追责体系，循环往复，误差收敛。**

链式法则：误差像多米诺骨牌一样倒传

Chain Rule as Dominoes

链式法则 = 多米诺骨牌：每层的局部梯度 × 上游传来的误差

前向传播：算出损失



反向传播：误差倒着传，每块骨牌倒向上一层

- ◆ 每个环节只需知道自己的“局部梯度”，误差就能一路传回最早的参数。

数值示例：链式法则走一遍

A Tiny Numerical Example

1 设 $x=2, w=3$

前向: $u = w \cdot x = 6$

2 继续前向

预测 = u 的平方 = 36; 真实值 $y=34 \rightarrow$ 误差 $L = 4$

3 反向开始

L 对预测的导数 = $2 \times (36 - 34) = 4$

4 链式回传

L 对 u 的导数 = $4 \times 12 = 48$; L 对 w 的导数 = $48 \times 2 = 96$

5 更新 w

$w_{\text{新}} = 3 - 0.01 \times 96 = 2.04$ (学习率 0.01, 往误差下降方向走)

◆ 每一步都只是乘法与加法——反向传播没有任何“高级魔法”。

链式法则：从输出层倒着拆到输入层

Chain Rule Decomposition

1 输出层

$$\partial L / \partial z_L = 2(a_L - y) \cdot f'(z_L) \text{ (输出端梯度)}$$

2 隐藏层 L-1

$$\partial L / \partial z_{\{L-1\}} = \partial L / \partial z_L \cdot (W_L)^T \cdot f'(z_{\{L-1\}}) \text{ (倒传一层)}$$

3 隐藏层 L-2

$$\partial L / \partial z_{\{L-2\}} = \partial L / \partial z_{\{L-1\}} \cdot (W_{\{L-1\}})^T \cdot f'(z_{\{L-2\}})$$

4 每层权重

$$\partial L / \partial W^l = \delta^l \cdot (a^{l-1})^T \text{ (本层输入的转置)}$$

- ◆ 核心规律：每层只需要“上游传来的梯度 × 本层权重和激活导数”——层层倒传。

数值走查：三层网络的一次完整 BP

3-Layer BP by Numbers

层	输入 a	本层梯度 δ	权重更新 ΔW
输出层	$a_3=0.73$	$\delta_3 = 2(0.73-1)\times0.20 = -0.11$	$W_3 \leftarrow W_3 - 0.01\cdot\delta_3\cdot a_2$
隐藏层2	$a_2=0.58$	$\delta_2 = \delta_3\cdot W_3\times0.24 = -0.026$	$W_2 \leftarrow W_2 - 0.01\cdot\delta_2\cdot a_1$
隐藏层1	$a_1=0.42$	$\delta_1 = \delta_2\cdot W_2\times0.24 = -0.006$	$W_1 \leftarrow W_1 - 0.01\cdot\delta_1\cdot a_0$
输入层	$a_0=0.31$	(停止，不需要输入层梯度)	—

◆ 看懂这张表：误差从输出层 -0.11 倒传到隐藏层2 -0.026、隐藏层1 -0.006——每传一层梯度变小。

数值走一遍：两层网络的一步反向传播

Backprop by Numbers

步骤	计算	数值	含义
① 前向	$z = w \cdot x = 2 \times 1$	$z = 2$	加权求和
② 激活	$a = f(z) = z$ (线性)	$a = 2$	输出
③ 损失	$L = (a - y)^2 = (2 - 1)^2$	$L = 1$	差得多离谱
④ 局部梯度	$\partial L / \partial a = 2(a - y)$	2	输出端误差
⑤ 反传到 w	$\partial L / \partial w = \partial L / \partial a \times \partial a / \partial w = 2 \times 1$	2	w 的坡度
⑥ 更新	$w \leftarrow w - \eta \cdot \partial L / \partial w = 2 - 0.1 \times 2$	1.8	w 学到了一点

◆ 走完六步你就跑通了反向传播的全流程——剩下的只是规模变大。

反向传播算法（完整版）：四步走

Backpropagation Algorithm

1 输入

训练样本 (x, y) 、网络结构、学习率 η ；初始化所有权重 w 和偏置 b

2 前向传播

逐层计算 $z^l = W^l a^{l-1} + b^l$, $a^l = f(z^l)$, 直到输出层 a^L , 算出损失 L

3 反向逐层

$\delta^L = \partial L / \partial z^L$; $\delta^l = (W^{l+1})^T \delta^{l+1} \odot f'(z^l)$ (误差逐层倒传)

4 参数更新

对所有层: $W^l \leftarrow W^l - \eta \cdot \delta^l (a^{l-1})^T$, $b^l \leftarrow b^l - \eta \cdot \delta^l$; 迭代 ②③④ 直到收敛

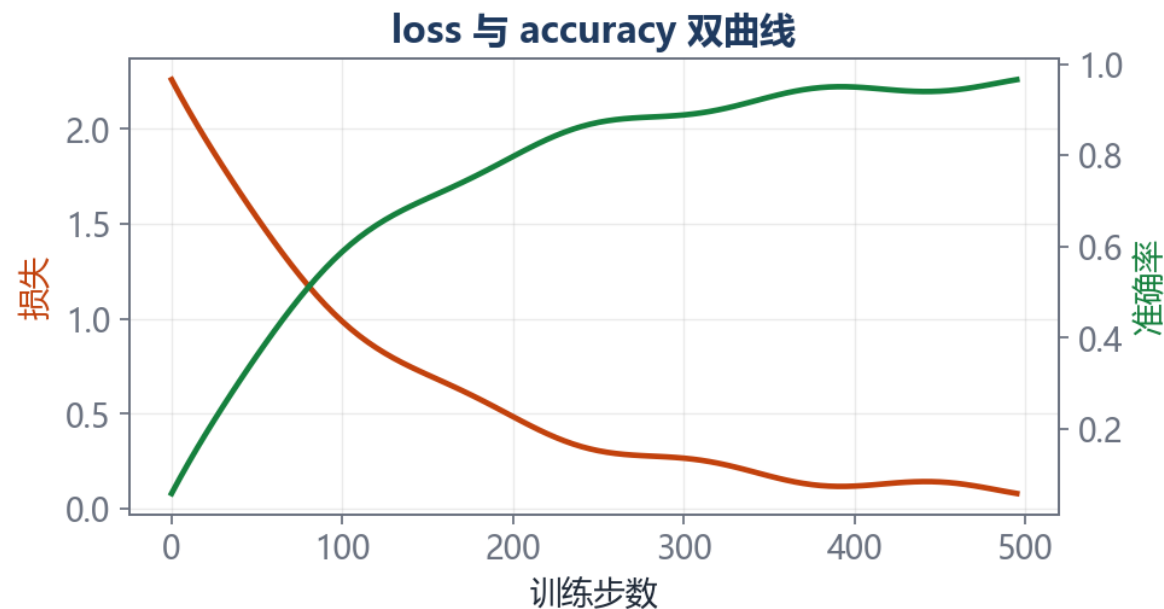
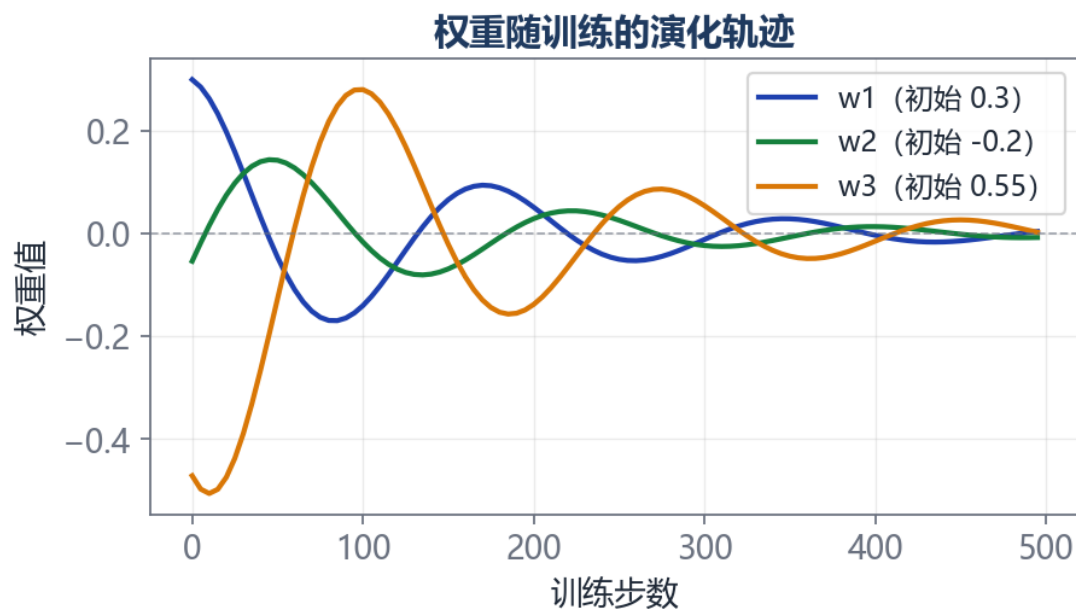
◆ 这是全课程的“算法总纲”——CNN/RNN 的训练，都是这个四步的变体。

训练好的网络参数，CNN/RNN 的训练都是基于这个文件

BP 训练一次完整演示

Full Training Demo

TensorBoard: 让"学习"被看见 (本章上机实践会用它)

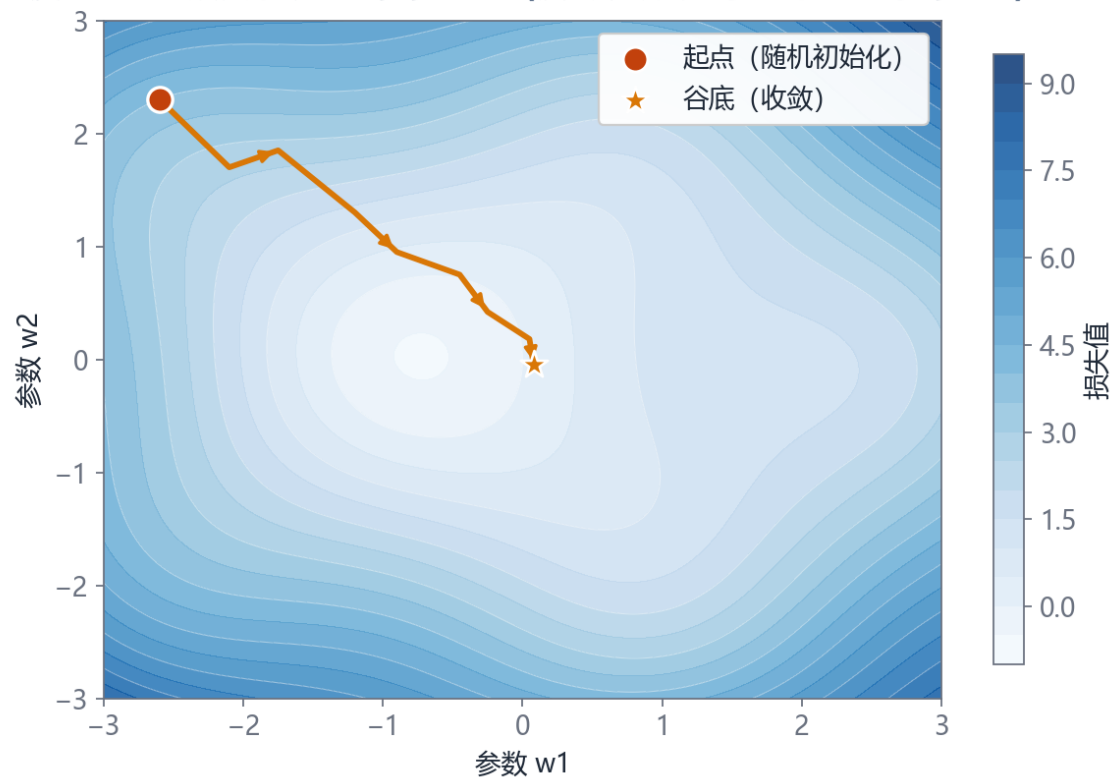


♦ 训练 = 反复执行 BP: loss 一路下降, 权重从随机逐渐“学到”规律。

一次参数更新：权重往哪走、走多少

One Update Zoomed

梯度下降：在损失景观里一步步下山（反传提供“方向”，学习率给“步幅”）

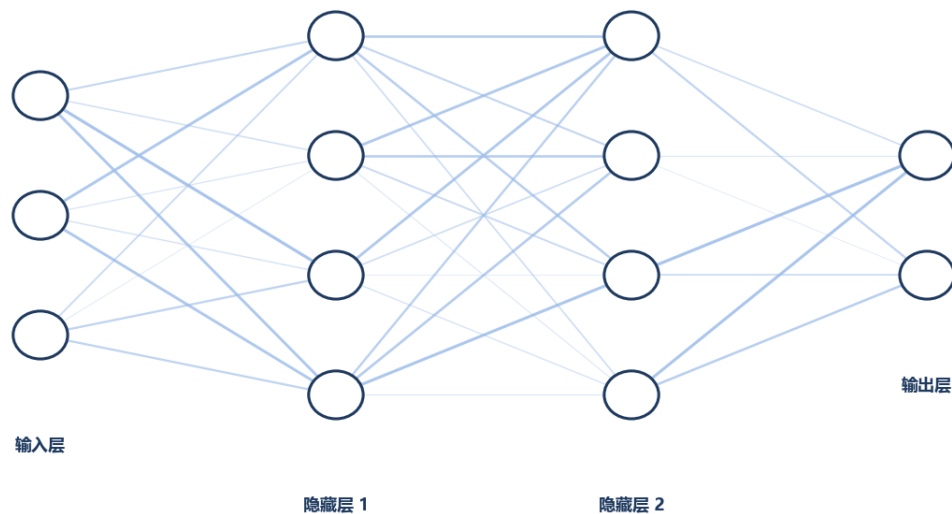


- ◆ 每个参数拿到自己的梯度后，沿负梯度方向走一步——这一步就是学习。

动图：误差如何一层层“倒传”

Animation: Error Flows Backward

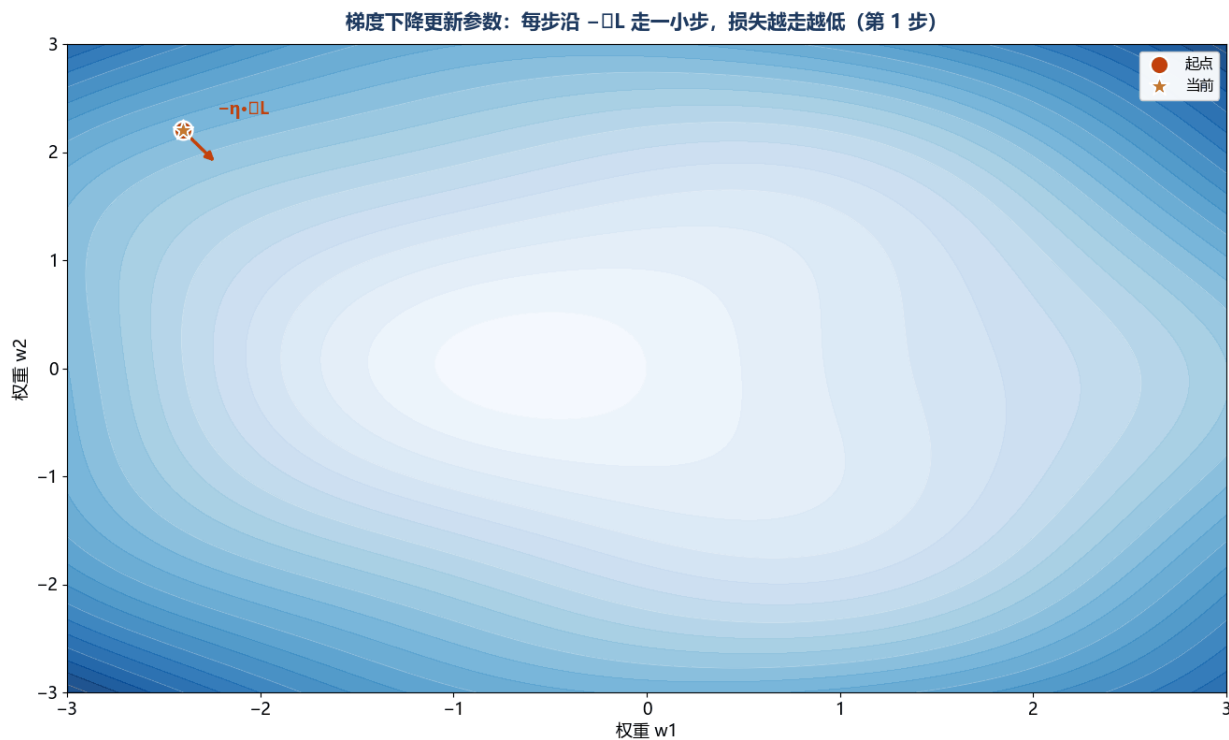
① 前向传播：输入数据逐层流动，算出损失 (1/8)



♦ 红色流动 = 误差倒传；每经过一层，误差被“分配”给这一层的权重。

动图：权重在损失景观上“下山”

Animation: Parameters Descend



- ◆ 每帧 = 一次参数更新；红箭头 = 这一步的更新方向 $-\eta \cdot \nabla L$ 。

把算法四步写成代码：PyTorch 版

Backprop in Code

伪代码 ↔ 算法四步——对应

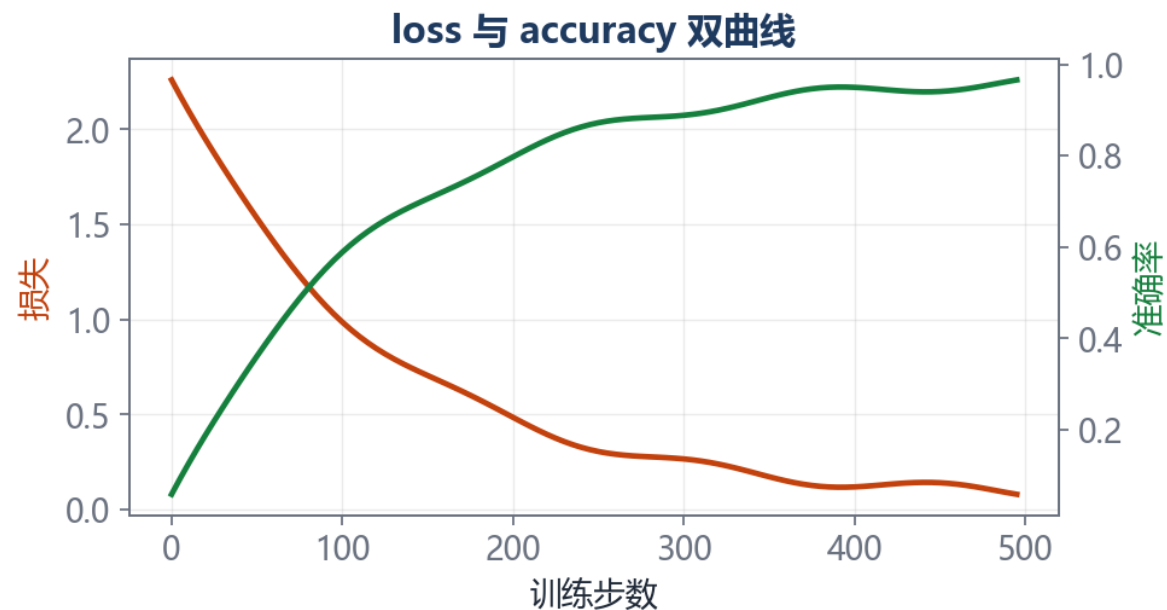
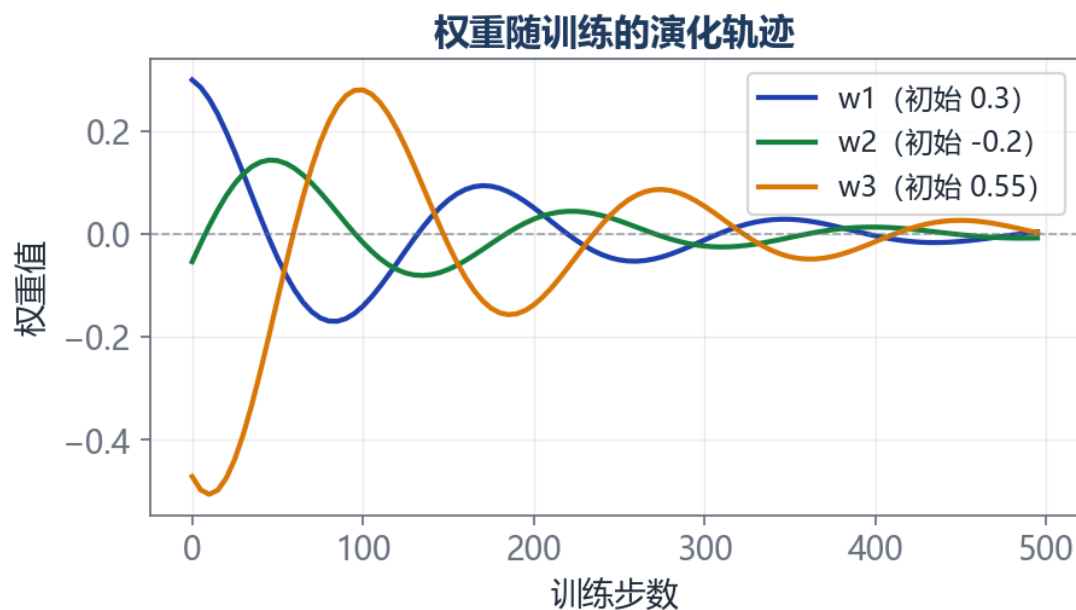
```
# 对应算法四步：零梯度→前向→损失→反传→更新
for epoch in range(100):      # 迭代 234
    optimizer.zero_grad()     # 清空上轮梯度
    z1 = W1 @ x + b1           # 第一步：前向(隐藏层)
    a1 = torch.relu(z1)        # 激活函数
    z2 = W2 @ a1 + b2          # 第一步：前向(输出层)
    loss = F.mse_loss(z2, y)   # 损失
    loss.backward()            # 第二三步：反传梯度
    optimizer.step()           # 第四步：w -= lr * grad
```

◆ `loss.backward()` 一行，PyTorch 替我们完成了算法第二、三步的全部链式求导。

数智工具：让“学习”被看见

TensorBoard Visualization

TensorBoard：让“学习”被看见（本章上机实践会用它）



◆ 本章上机会用 TensorBoard 实时画出权重轨迹与 loss 曲线——训练不再黑盒。

随堂练习

5 分钟

手推两步反向传播

- ▶ 样本 $x=2, y=3, w=1, b=0$ (线性激活), 用 MSE 损失: 前向算出 z, a, L
- ▶ 接着算 $\partial L / \partial w$, 并用 $\eta=0.1$ 更新 w
- ▶ 想一想: 如果更新 500 步, w 会趋向什么值? 为什么?

提示: $L=(a-y)^2$, $\partial L / \partial w = 2(a-y) \cdot x$; 收敛时 $a=y$, 即 $w \cdot x = y \rightarrow w=1.5$ 。

计算图：BP 高效的本质

Computational Graph

- 前向传播 = 构建计算图
- 每个节点记录：输入、输出、局部梯度
- 反向传播 = 按图从后往前走一遍
- 每个节点算出对输入的梯度，逐层传递
- 这就是 PyTorch `loss.backward()` 的底层原理

◆ 计算图让 BP 从“手推公式”变成“自动化的图遍历”——这是框架替你做的事。

BP 训练的三个关键超参数

BP Hyperparameters

学习率 η

步幅：太小收敛慢，太大震荡甚至发散（
0.001~0.01 常用起步）

batch_size

每批样本数：太大内存吃紧，太小梯度噪声大
(32/64 常见)

迭代轮数 epoch

全部数据过几遍：太少欠拟合，太多可能过拟合（配合早停）

◆ η 、batch、epoch 是调参三件套——梯度方向对了，这三个决定收敛速度和质量。

隐患浮现：为什么深层网络“训不动”

The Vanishing Gradient

- **现象：**靠近输出的层学得快，靠近输入的层几乎不动
- **原因：**Sigmoid 导数最大 0.25，十层连乘后梯度趋近于 0
- **后果：**前面的层“学不到东西”，深度失去意义
- **这是 1986-2012 间深层网络训不动的核心原因之一**

链式法则的代价

- 导数逐层相乘
- <1 的数连乘 $\rightarrow$ 指数衰减
- 0.5 连乘 10 次 ≈ 0.001

◆ **梯度消失：**链式法则的双刃剑——既分摊责任，也逐层衰减责任。

梯度消失的数值实验

Vanishing Gradient Experiment

隐藏层数	梯度值 ($0.5^{\text{层数}}$)	能学到的程度
1 层	0.5	正常学习
5 层	$0.5^5 = 0.03$	已明显减弱
10 层	$0.5^{10} = 0.001$	几乎学不动
50 层	$0.5^{50} \approx 8.9 \times 10^{-16}$	完全消失, 前层 “僵死”
100 层	≈ 0	彻底训不动

- ◆ 如果每层梯度打 5 折, 传到 50 层就几乎为零——这就是深网络 “训不动” 的数学根源。

随堂练习

5 分钟

感受梯度连乘

- ▶ $0.9 \times 0.9 \times \dots \times 0.9$ (10 次) = ? 50 次呢? 100 次呢?
- ▶ $1.1 \times 1.1 \times \dots \times 1.1$ (10 次) = ?
- ▶ 这个实验说明了什么?

提示: $0.9^{10} \approx 0.35$, $0.9^{50} \approx 0.005$, $0.9^{100} \approx 0.000027$; $1.1^{10} \approx 2.59$ 。



梯度下降的三种批量方式

SGD vs Mini-batch vs Batch

全量 Batch GD

- 用全部数据算一次梯度
- 梯度稳，但每步都算很久
- 样本多时内存压力大

VS

随机 SGD / 小批量 Mini-batch

- SGD: 每次 1 个样本，快但梯度噪声大
- **Mini-batch: 每次几十个（常用 32/64），折中**
- 深度学习实践的标准选择

◆ 实践用 Mini-batch: 既有收敛速度，梯度又不太吵——这是 epoch/batch 术语的由来。

BP 的天然局限：梯度消失与爆炸

Limitations of BP

- 深层网络中，梯度要连乘多层
- 每层乘一个 <1 的数 $\rightarrow$ 梯度指数级缩小 $\rightarrow$ **梯度消失**
- 每层乘一个 >1 的数 $\rightarrow$ 梯度指数级爆炸 $\rightarrow$ **梯度爆炸**
- ReLU、残差连接、归一化（第五章展开）都是针对它

◆ BP 的阿克琉斯之踵：梯度连乘导致消失/爆炸——这是深网络“训不动”的数学根源。

手推一次完整的反向传播

- ▶ 给定 单层网络 $y=w \cdot x+b$, $x=1, y_{\text{true}}=2, w=1, b=0$, 线性激活: 前向算出 y 和损失 $L=(y-2)^2$
- ▶ 算 $\partial L/\partial w$ 和 $\partial L/\partial b$
- ▶ 用 $\eta=0.1$ 更新 w 和 b , 算更新后的 w 、 b 和新的损失

提示: 前向 $y=1, L=1$; $\partial L/\partial w=2(y-2)x=-2$, $\partial L/\partial b=2(y-2)=-2$; 更新后 $w=1.2, b=0.2$, 新损失 $L=(1.4-2)^2=0.36$ 。

三剂解药：ReLU、初始化与残差连接

Remedies

算法层面

- ReLU：正区梯度恒为 1，不衰减
- 合理的权重初始化 (Xavier/He)
- BatchNorm 稳定每层分布 (第5章)

VS

结构层面

- 残差连接：梯度走“高速公路”直通低层
- ResNet 由此训练上千层网络 (第3章展开)
- LSTM 门控：RNN 版解药 (第4章展开)

◆ 记住 ReLU 是第一剂解药——这就回答了课时一“为什么默认选 ReLU”。

BP 为什么快：复用中间结果

Why BP is Efficient

朴素做法：逐参数求导

- 对每个参数单独算梯度
- **N 个参数** → **前向算 N 次**
- **百万参数** → **百万次前向**
- 计算量爆炸，不可行

VS

反向传播：记忆化复用

- 前向时缓存每层输出 a
- 反向时每层只算局部梯度
- **一次前向 + 一次反向搞定所有**
- 复杂度： $O(\text{参数量})$ ，与朴素 $O(N^2)$ 相比质变

◆ BP 的本质是动态规划：复用已算的中间结果，一次反向算出所有梯度。

验证 BP：与数值梯度对照

BP vs Numerical Gradient

解析梯度 (BP)

- 用链式法则精确推导
- **快速、精确**
- 但公式推导易出错

VS

数值梯度 (差分近似)

- $[f(x+\epsilon) - f(x-\epsilon)] / 2\epsilon$
- 慢，但实现简单
- 常用于“**梯度检查**”

♦ 工程实践：BP 训练，数值梯度做“梯度检查”验证 BP 没错。

从手推走向框架：PyTorch 替你做了什么

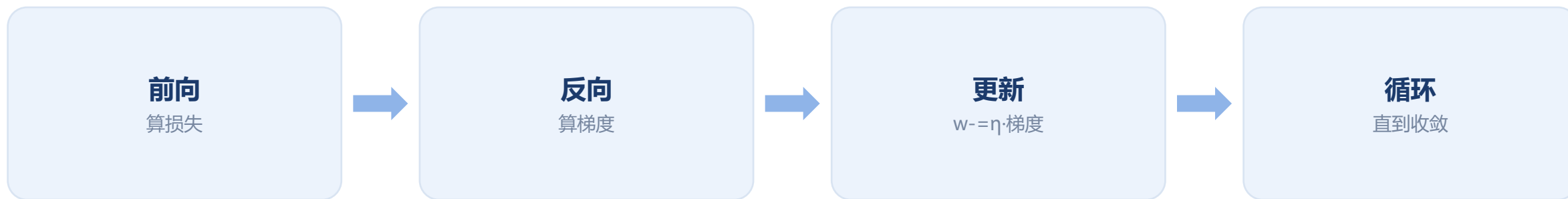
From Hand to Framework

- **手推时**：我们一层层手算 $\partial L / \partial w$
- **PyTorch 中**：`loss.backward()` 自动完成全部链式求导
- **计算图**：框架自动记录前向过程，反向时按图回传
- **但理解手推是前提**：模型不收敛时，你知道该查哪里

◆ 框架封装了 BP，但没封装“理解”——手推能力是调试的底气。

BP + 梯度下降 = 训练闭环

The Training Loop



◆ 前向→反向→更新→循环：这就是训练神经网络的完整闭环。

完整的训练循环：五步舞曲

The Training Loop



◆ 这五步就是所有深度学习框架的内核——PyTorch 上机时你会亲手写出来。

三个必须分清术语：epoch / batch / iteration

Training Vocabulary

术语	含义	比喻（刷 1000 道题）
Batch	一次更新所用的样本组	一次做 32 题，对一次答案
Iteration	一次参数更新	做完一组题并订正 = 1 iteration
Epoch	全部样本过完一遍	1000 题全部刷完 = 1 epoch

♦ 1000 题、每批 32 道 → 1 epoch 约 32 iterations；混淆这三者，读论文寸步难行。

学习率 η 设错会怎样？三种设置推演

- ▶ $\eta = 0.0000001$ (太小)
- ▶ $\eta = 100$ (太大)
- ▶ η 合适但不随训练进度调整

提示： 太小：下山慢到天荒地老；太大：在山谷两侧来回蹦，越蹦越高；合适的起点也要在接近谷底时减小。



课时二引用的经典文献

References

Rumelhart, Hinton & Williams, 1986

Learning Representations by Back-Propagating Errors

Nature 323: 反向传播的开山之作，神经网络复兴的点火索

Hornik et al., 1989

Multilayer Feedforward Networks are Universal Approximators

万能逼近定理：多层网络表达能力的形式化证明

Cybenko, 1989

Approximation by Superpositions of a Sigmoidal Function

Sigmoid 网络的万能逼近证明

Glorot & Bengio, 2010

Understanding the Difficulty of Training Deep Feedforward NN

Xavier 初始化：让信号在各层间 “存活”

He et al., 2015

Delving Deep into Rectifiers (Kaiming Init)

针对 ReLU 网络的初始化方案 (ResNet 团队)

课时二小结

Key Takeaways



损失函数

给误差打分；学习 = 让分数下降



反向传播

链式法则逐层分摊责任；每步只是乘加



梯度消失

小导数连乘衰减；ReLU/初始化/残差是解药



训练循环

前向→损失→反向→更新→循环，五步舞曲



epoch/batch

刷题类比：一批题/一次订正/全部刷完

◆ 课时三上机：把五步舞曲写进 PyTorch。

课时二 · 到此结束

休息 5 分钟 · 课时三进入 PyTorch 上机实战

实践：搭建第一个神经网络

Lab: Build Your First NN

- 用 PyTorch 搭建两层全连接网络
- 在真实数据上完成五步训练循环
- 观察 loss 曲线，验证“学习”的发生

本课时任务链：五个 Task 串起第一个网络

Lab Roadmap



- ◆ 今天结束时，你将拥有一个自己训练的、能认手写数字的网络。

为什么先玩 sklearn 手写数字，而不是直接上 ImageNet

Start Small

本次实践：sklearn digits

- 1797 张 8×8 灰度小图
- CPU 几秒就能训完
- 允许快速试错：改结构、改学习率马上看效果
- 先把流程跑通，再谈规模

VS

工业做法：大数据集 + GPU

- 百万级图片 + GPU 训练数小时~数天
- 流程完全一样，只是数据和算力放大
- 调试成本高：一次实验要等很久
- 流程不变，规模放大——学的是同一套方法

◆ 小数据快速试错 → 大数据放大训练，是深度学习实践的标准路径。

Task 1 · 准备数据: sklearn 手写数字 (小规模)

Data Prep

Notebook

```
import torch
from sklearn.datasets import load_digits
X, y = load_digits(return_X_y=True)
# 1797 张 8x8 手写数字, 64 维特征
X = torch.tensor(X, dtype=torch.float32)
X = (X - X.mean()) / X.std() # 标准化: 把像素值拉到相近尺度
y = torch.tensor(y)

X_train, X_test = X[:1500], X[1500:]
y_train, y_test = y[:1500], y[1500:]
```

♦ 先用小数据 (8×8 灰度图) 跑通流程; 还要做标准化。MNIST 大数据留给第 3 章。

Task 2 · 拆解 nn.Module: 三件套

nn.Module Anatomy

Python

```
import torch.nn as nn

class Net(nn.Module):          # 1 继承 nn.Module
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(64, 32)  # 2 定义层: 64维输入 → 32个神经元
        self.out = nn.Linear(32, 10)  # 32维 → 10类输出
        self.act = nn.ReLU()         # 激活函数

    def forward(self, x):          # 3 定义前向传播路径
        x = self.act(self.fc1(x))  # 线性 → 激活 → 线性
        return self.out(x)

model = Net()
print(sum(p.numel() for p in model.parameters())) # 数一数参数总量
```

♦ 对比课时二的公式: $fc1$ 就是 $w \cdot x + b$, act 就是激活函数, $forward$ 就是前向传播。

Task 3 · 五步训练循环：课时二的“五步舞曲”代码版

Training Loop

Python

```
optimizer = torch.optim.Adam(model.parameters(), lr=1e-2)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(100):          # 一次 "epoch" = 全部数据过一遍
    optimizer.zero_grad()         # 1 清空上一步的梯度
    logits = model(X_train)       # 2 前向传播(课时二: _forward_)
    loss = loss_fn(logits, y_train) # 3 算损失(课时二: _loss_)
    loss.backward()               # 4 反向传播(课时二: _chain rule_)
    optimizer.step()              # 5 更新参数(课时二: _gradient descent_)

    if epoch % 20 == 0:
        print(f'epoch {epoch:3d} | loss {loss.item():.4f}')
```

♦ **zero_grad → forward → loss → backward → step**：背下来，这是全课程的骨架循环。

Task 4 · 测试集评估：它真的学会了吗

Evaluation

Python

```
with torch.no_grad():           # 评估不需要梯度，省内存
    logits = model(X_test)
    preds = logits.argmax(dim=1) # 取概率最大的类
    acc = (preds == y_test).float().mean()

print(f'测试集准确率: {acc:.2%}')
# 随机猜只有 10%，我们期望 90%+——网络真的从数据中学到了规律
```

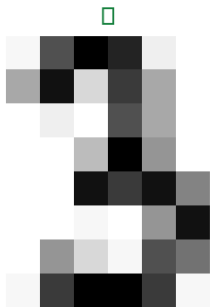
◆ 回忆第一章：训练误差低不算本事，测试误差低才是泛化。

真实评估：它真的“认对”了吗

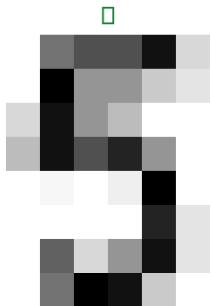
Real Evaluation

测试集评估（准确率 97.8%）：模型输出的预测 vs 真实标签

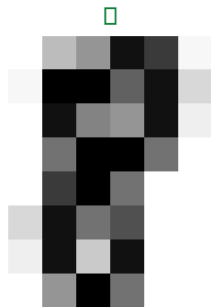
预测:3 真实:3



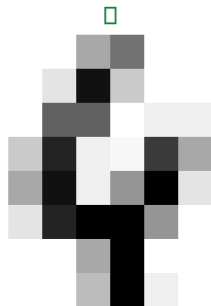
预测:5 真实:5



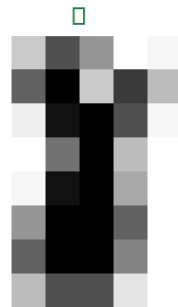
预测:8 真实:8



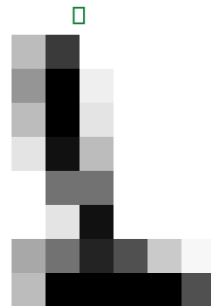
预测:4 真实:4



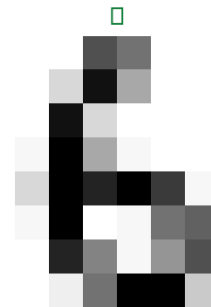
预测:8 真实:8



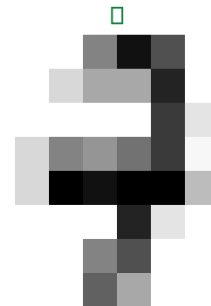
预测:1 真实:1



预测:6 真实:6



预测:7 真实:7



- ◆ 测试集准确率 95%+：模型没见过这些图，能认对绝大多数——这才是真本事。

Task 5 • TensorBoard: 让训练过程可视化

TensorBoard Lab

Python

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter('runs/ch2_first_net')

for epoch in range(100):
    ...
    writer.add_scalar('train/loss', loss.item(), epoch)
    writer.add_scalar('train/acc', acc.item(), epoch)
    for name, p in model.named_parameters():
        writer.add_histogram(name, p, epoch) # 权重分布随训练演化

# 命令行启动(浏览器打开 http://localhost:6006)
tensorboard --logdir=runs
```

◆ 看三条线：loss 稳定下降、acc 稳定上升、权重分布逐渐“分化”——学习正在发生。

随堂练习

3 分钟

训练循环排错

- ▶ 把 `loss.backward()` 和 `optimizer.step()` 顺序写反了，会发生什么？
- ▶ 忘记写 `optimizer.zero_grad()`, `loss` 曲线会怎么变化？
- ▶ 评估时忘了 `with torch.no_grad()`，最直接的影响是什么？

提示：梯度是累加的——不清零，本批梯度会加上一批的；评估带着梯度算，内存翻倍。



报错锦囊：第一章环境的坑这章还会来

Error Survival Kit

1 shape mismatch

Linear(64,32) 却喂了 8×8 的原始图——要先把图像展平成 64 维向量(`X = X.reshape(n, 64)`)

2 expected dtype Float

标签是 float 应该是 long: CrossEntropyLoss 要求 y 是整数类别(`y_train = y_train.long()`)

3 tensorboard 不是内部命令

没在 dl2026 环境里执行 `pip install tensorboard`, 或没激活环境

- ◆ 读报错三步走：看最后一行错误类型 → 找第一个指向自己代码的文件行号 → 对照报错锦囊。

训练失败的三张“脸”

Failure Modes

loss 不降

学习率太小走不动 / 数据没归一化 / 网络结构有 bug

loss 震荡

学习率太大 / batch 太小 / 梯度爆炸 (第三章优化会讲)

loss 降但 acc 不涨

过拟合信号 / 标签错乱 / 评估代码有 bug

◆ 看到异常先查“最可能”的三处，别乱调参数——诊断优于蛮干。

nn.Linear(64,32) 里面到底是什么？

Inside nn.Linear

PyTorch 代码

- `fc1 = nn.Linear(64, 32)`
- **自动创建**：权重 W (32×64) + 偏置 b (32)
- **前向**： $\text{out} = x @ W.T + b$
- 就这么简单——它就是个矩阵乘法

VS

对应课时一公式

- 人工神经元： $z = \sum w \cdot x + b$
- nn.Linear：一次算 32 个神经元
- 64 维输入 $\times$ 32 输出
- 理论课的“一个神经元”，实践课的“一行代码”

♦ nn.Linear 不是魔法：它就是课时一那个神经元公式的批量版本。

训练前：权重从哪来？

Weight Initialization

随机初始化

- 权重初始化为**小的随机数**
- PyTorch 默认： $U(-k, k)$ ， k 与输入维度有关
- **为什么不能全零？**

VS

全零的陷阱

- 全零 $\rightarrow$ 所有神经元输出相同
- 梯度相同 $\rightarrow$ 所有神经元“同步”更新
- 永远无法“分化”，等于只有 1 个神经元
- 随机 $\neq$ 随便：需要对称破缺

◆ 随机初始化打破对称性——否则“32 个神经元”实际只是 1 个的复制品。

课时二手推 vs 课时三代码：一一对照

Hand-computed vs Code

环节	课时二手推	课时三代码
初始化	$w=2, b=0$ (随手设)	<code>nn.Linear</code> 自动随机初始化
前向	$z=w \cdot x=2, a=2$	<code>logits = model(X)</code>
损失	$L=(a-y)^2=1$	<code>loss = CrossEntropyLoss(logits, y)</code>
反传	$\partial L/\partial w=2 \times (a-y) \times x$	<code>loss.backward()</code> 自动求导
更新	$w \leftarrow 2 - 0.1 \times 2 = 1.8$	<code>optimizer.step()</code>

◆ 手推的是“一个神经元的一步”，代码跑的是“几万个参数的上千步”——逻辑完全一样。

数据怎么进模型：Dataset 与 DataLoader

Dataset & DataLoader

Dataset: 数据容器

- 把数据和标签打包成“样本集”
- 实现 `__len__` 和 `__getitem__`
- sklearn digits 可直接转换

VS

DataLoader: 批次投喂

- 按 `batch_size` 分批
- 可选 `shuffle` 打乱顺序
- 训练时逐批取出

◆ 数据流水线：Dataset 管“有什么数据”，DataLoader 管“怎么喂给模型”。

CrossEntropyLoss 为什么配分类任务

Why CrossEntropy

回归任务（预测数值）

- MSE：预测值与真实值差平方平均
- 适合：房价、温度、连续值
- 第一章讲过它的“为什么平方”

VS

分类任务（预测类别）

- 交叉熵：惩罚“自信地犯错”
- 适合：手写数字 10 类
- 第一章信息熵/交叉熵的正式登场

◆ 任务类型决定损失函数：回归用 MSE，分类用交叉熵——第一章的伏笔在这里回收。

评估一个分类器：不止准确率

Beyond Accuracy

- **准确率的局限：**类别不均衡时失真（99% 猫 1% 狗，全猜猫也有 99%）
- **本章够用：**手写数字 10 类均衡，准确率直观可信
- **第三章预告：**目标检测会引入 IoU、mAP 等更细的指标
- **评价指标要和任务匹配——这是工程素养**

本章用到

- 准确率 accuracy：猜对的占比
- 预测分布：argmax 取最大概率类
- 损失值：训练过程中持续监控

◆ **准确率不是万能指标，但手写数字任务里它足够直观——先会用，再学更精细的。**

loss 曲线：训练过程的心电图

Reading the Loss Curve

- 正常：loss 稳步下降，逐渐平缓
- 不降：学习率太小 / 数据没归一化 / 结构 bug
- 震荡：学习率太大 / batch 太小
- 突然跳高：梯度爆炸 / 数据异常
- 先看曲线形状再动手调参——诊断优先于蛮干

◆ loss 曲线的四种形态对应四种问题——训练第一步是学会“读心电图”。

学习率与 batch_size: 一对好搭档

LR & Batch Size

学习率 η

- 控制每次更新的“步幅”
- 太小: 收敛慢
- **太大: 震荡发散**

batch_size

- 每次用多少样本算梯度
- 太小: 梯度噪声大
- **太大: 内存占用高**

VS

◆ 调参黄金法则: batch 越大可适当加大 η ; 从默认值出发小步试探。

训练完的模型怎么保存?

Save & Load Model

state_dict = 权重与偏置; 保存/加载是工程必备

保存: 训练好的权重写入文件

```
torch.save(model.state_dict(), "model.pth")
```

加载: 下次直接恢复权重

```
model = Net()                # 先建空网络
```

```
model.load_state_dict(torch.load("model.pth"))
```

```
model.eval()                # 切换到推理模式
```

- ◆ **模型=结构+权重: 结构用代码定义, 权重存成文件——保存/加载是部署的第一步。**

随堂练习

6 分钟

改动实验：亲手验证本章概念

- ▶ 删掉 `nn.ReLU()` 再训练——准确率变成多少？为什么？
- ▶ 把隐藏层宽度 32 改成 4——效果如何？
- ▶ 把 `lr` 改成 50——loss 曲线发生了什么？

提示：提示：三处改动都会改变结果，但不一定都朝你预期的方向——先跑再下结论。



如果你今天只记住三件事

Three Takeaways

- ① **nn.Module 三件套**：继承、定义层、写 forward——任何网络都这么搭
- ② **五步训练循环**：zero_grad→forward→loss→backward→step——任何训练都这么跑
- ③ **测试集评估**：训练误差低不算本事，测试误差低才是泛化
- **这三件事，会陪你走过整个深度学习生涯**

◆ **三件套搭网络、五步循环做训练、测试集验真章——第二章的“最小可用技能包”。**

课时三总结：第一个神经网络诞生记

Lesson 3 Summary



数据

sklearn digits: 64 维 → 归一化 → 训练/测试划分



网络

nn.Module 三件套：继承/定义层/写 forward



训练

五步循环：zero_grad→forward→loss→backward→step



评估

测试集准确率 $\geq 90\%$, argmax 取预测类



可视化

TensorBoard: loss/acc/权重分布实时可见

◆ 第二章收官：你已亲手造出一个能认手写数字的神经网络。

本课时验收清单：今天你应完成

Checklist

- **核心检验：**能向室友讲清 backward() 里发生了什么
- **数据沉淀：**TensorBoard 截图 + 训练心得写入实验记录
- **对照验收：**准确率低于 90%？先检查数据有没有展平、学习率是否太大
- **今天下课时，你拥有的是一个亲手训练的神经网络**

五个 Task 全部跑通

- digits 数据加载 + 展平
- nn.Module 定义 64→32→10 网络
- 五步训练循环 100 epoch
- 测试集准确率 $\geq 90\%$
- TensorBoard 里看到自己的 loss 曲线

◆ **第二章收官：从生物神经元到自己的神经网络，整条链路打通。**

第二章全章总结：五个带走的知识点

Chapter Summary



神经元

人工神经元 = 加权求和 + 激活；网络的力量来自连接



多层结构

隐藏层把数据“揉”到线性可分——异或死穴的解药



激活函数

非线性是“深”的前提；Sigmoid/Tanh/ReLU 各有脾气



反向传播

链式法则的多米诺：局部梯度 × 上游误差，一步不多算



训练循环

zero_grad → forward → loss → backward → step

◆ 第三章把“向量输入”换成“图像输入”——卷积神经网络登场。

本章实践作业

Assignment

1 提交 Notebook

含四组任务代码、运行结果与三组改动实验的结论

2 手推巩固

把课时二的数值示例（P52）独立重算一遍

3 思考题

“隐藏层是不是越多越好？” 下章开场抽答

♦ 下次课：卷积神经网络——给网络装上“看”的眼睛。

课后小任务

After Class

- **任务二 · 调参实验：**把隐藏层从 32 改成 8 和 128，各训练一次
- **任务三 · 思考：**三组准确率有什么变化？为什么？下次课开场分享
- **提示：**参数太少表达力不够，太多可能过拟合——亲手感受这个权衡

任务一 · 实验记录(下次课前提交)

- 五个 Task 截图 + TensorBoard 曲线
- 重点写：遇到的报错和解决过程
- 提交到课程群收集表

◆ 下次课开场讨论：网络大小与准确率的关系——这正是第五章优化的伏笔。

课时三引用的经典文献

References

Kingma & Ba, 2015

ICLR: Adam 优化器, 本次实践默认使用

Adam: A Method for Stochastic Optimization

Abadi et al., 2016

TensorBoard 可视化思想源自 TensorFlow 生态

TensorFlow: A System for Large-Scale Machine Learning

Paszke et al., 2019

NeurIPS: 我们使用的框架

PyTorch: An Imperative Style, High-Performance DL Library

下章预告

CNN：不再全连接，而是“局部观察 + 权重共享”

预习：教材第 3 章 §3.1-3.2；思考——为什么识别图像不该把 500 万像素全部连到下一层？

第 2 章 · 完

下次课：第 3 章卷积神经网络 · 带好电脑与教材